

**UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA
DE COLOMBIA**

UPTC - FACULTAD SECCIONAL SOGAMOSO

**ESCUELA DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN**

MATEMÁTICAS DISCRETAS

TALLER PRÁCTICO FINAL

**Verificación, Validación y Análisis Experimental
de Algoritmos de Ordenamiento en C#**

Manual de Usuario y Documentación Técnica

Integrantes del grupo:

[Nombre del estudiante 1] - Código: [código]

[Nombre del estudiante 2] - Código: [código]

[Nombre del estudiante 3] - Código: [código]

Fecha de entrega: 5 de junio de 2025

Docente: [Nombre del docente]

TABLA DE CONTENIDO

1. Introducción	3
2. Requisitos del Sistema	3
3. Instalación y Configuración	4
4. Manual de Usuario	5
4.1. Pantalla de Configuración	5
4.2. Ejecución de Pruebas	6
4.3. Visualización de Datos Generados	7
4.3. Tabla de Métricas Detalladas	8
4.4. Estadísticas Consolidadas	9
4.5. Gráficos Comparativos	10
5. Descripción Técnica	11
5.1. Algoritmos Implementados	11
5.2. Métricas Operacionales	13
5.3. Validación Automática	14
6. Respuestas a Preguntas del Taller	15
7. Conclusiones	22
8. Anexos	23

1. INTRODUCCIÓN

Este documento presenta el manual de usuario y la documentación técnica de la aplicación desarrollada para el Taller Práctico Final de la asignatura Matemáticas Discretas. La aplicación permite realizar un análisis experimental comparativo de diez algoritmos de ordenamiento, midiendo tiempos de ejecución, métricas operacionales y validando automáticamente la correctitud de cada ordenamiento.

El objetivo principal es relacionar los conceptos teóricos de complejidad algorítmica con resultados empíricos, permitiendo observar cómo diferentes tipos de datos (aleatorios, parcialmente ordenados, descendientes y con alta repetición) afectan el desempeño de cada algoritmo.

2. REQUISITOS DEL SISTEMA

Requisitos mínimos:

- Sistema operativo: Windows 7 SP1 o superior (Windows 10/11 recomendado)
- .NET Framework 4.8 (o 4.7.2)
- Procesador: 1.8 GHz o superior
- Memoria RAM: 4 GB mínimo (8 GB recomendado)
- Espacio en disco: 100 MB libres
- Visual Studio 2019/2022 (Community Edition es suficiente) para compilar

Requisitos de software:

- Microsoft Visual Studio 2019 o 2022
- Windows Forms App (.NET Framework) como tipo de proyecto
- Referencia a System.Windows.Forms.DataVisualization

3. INSTALACIÓN Y CONFIGURACIÓN

Paso 1: Descomprimir el archivo ZIP del proyecto en una carpeta local.

Paso 2: Abrir Visual Studio y seleccionar *Archivo* → *Abrir* → *Proyecto/Solución*.

Paso 3: Navegar hasta la carpeta descomprimida y seleccionar el archivo **OrdenamientoApp.csproj**.

Paso 4: Visual Studio cargará automáticamente todos los archivos del proyecto.

Paso 5: Verificar que el proyecto tenga como destino **.NET Framework 4.8** en las propiedades del proyecto.

Paso 6: Presionar **F5** o hacer clic en *Iniciar* para compilar y ejecutar la aplicación.

Nota importante: Si aparece algún error de referencia a `System.Windows.Forms.DataVisualization`, agregar la referencia manualmente desde *Referencias* → *Agregar Referencia* → *Ensamblados* → *System.Windows.Forms.DataVisualization*.

4. MANUAL DE USUARIO

La interfaz de la aplicación está organizada en cinco pestañas (TabControl) para evitar saturación visual y facilitar la navegación entre las diferentes funcionalidades.

4.1. PANTALLA DE CONFIGURACIÓN Y EJECUCIÓN

Esta es la pantalla principal donde el usuario configura los parámetros y controla todo el proceso experimental.

Parámetros de Entrada:

- **Valor mínimo (numericUpDownMin):** Límite inferior del rango de valores enteros. Puede ser negativo. Valor por defecto: -1000000.
- **Valor máximo (numericUpDownMax):** Límite superior del rango de valores enteros. Debe ser estrictamente mayor que el mínimo. Valor por defecto: 1000.

Validaciones automáticas: El sistema verifica que $\text{min} < \text{max}$. Si el rango excede 1.000.000, muestra una advertencia sobre Counting Sort y Radix Sort.

Controles de Ejecución:

- **Ejecutar Pruebas Completas (buttonEjecutar):** Inicia las 800 ejecuciones automáticas. Deshabilita los demás botones durante la ejecución y muestra una barra de progreso.
- **Ver Datos Generados (buttonVerDatos):** Navega a la pestaña de datos con los filtros seleccionados.
- **Generar Gráficos (buttonGenerarGraficos):** Crea gráficos comparativos de tiempos promedio.
- **Generar Estadísticas (buttonGenerarEstadisticas):** Calcula mínimo, máximo y promedio, y genera conclusiones automáticas.
- **Exportar Resultados (buttonExportarPDF):** Guarda todas las métricas y estadísticas en un archivo de texto.

Barra de Progreso:

Durante la ejecución, el progressBarEjecucion muestra el porcentaje completado y labelProgreso indica 'Progreso: X / 800'. La interfaz no se bloquea gracias a Application.DoEvents().

Conclusiones Automáticas:

El textBoxConclusiones muestra un análisis automático que incluye: mejor algoritmo por tamaño, algoritmos con buen desempeño al crecer, peor con datos descendentes, comparación Quick Sort vs Quick Sort 3 Vías, análisis de Counting Sort/Radix Sort, variación entre iteraciones, y mejor algoritmo para datos parcialmente ordenados.

4.2. EJECUCIÓN DE PRUEBAS

Al presionar 'Ejecutar Pruebas Completas', el sistema realiza automáticamente el siguiente diseño experimental:

- 4 tamaños de arreglo: 100, 1.000, 5.000, 10.000 elementos
- 5 iteraciones independientes por tamaño
- 4 tipos de datos por iteración: Aleatorio, Tendencia ascendente, Descendente, Alta repetición
- 10 algoritmos de ordenamiento por cada combinación
- **Total: $4 \times 5 \times 4 \times 10 = 800$ ejecuciones**

Cada algoritmo trabaja sobre una **copia independiente** del arreglo original, preservando la condición inicial de prueba. Los tiempos se miden con la clase **Stopwatch** en ticks y milisegundos.

Tipos de datos generados:

Aleatorio: Valores enteros distribuidos uniformemente entre min y max usando Random.Next().

Tendencia ascendente: Valores con tendencia general creciente pero con pequeñas variaciones aleatorias. Útil para simular datos parcialmente ordenados.

Descendente: Valores organizados de mayor a menor. Representa el peor caso para muchos algoritmos.

Alta repetición: Solo 5 valores distintos repetidos muchas veces. Permite comparar Quick Sort tradicional vs Quick Sort de tres vías.

4.3. VISUALIZACIÓN DE DATOS GENERADOS

La pestaña 'Datos Generados' permite inspeccionar los arreglos originales antes de ordenarlos.

Filtros disponibles:

- comboBoxTamano: Selecciona entre 100, 1000, 5000, 10000
- comboBoxIteracion: Selecciona entre iteración 1 a 5
- comboBoxTipoDatos: Selecciona el tipo de datos a visualizar

El dataGridViewDatos muestra una tabla con columnas: ID, Aleatorio, Tendencia ascendente, Descendente, Alta repetición. Se limita a 100 filas para no saturar la interfaz.

El chartDatos muestra un gráfico de líneas con los cuatro tipos de datos superpuestos, permitiendo comparar visualmente las distribuciones.

4.4. TABLA DE MÉTRICAS DETALLADAS

La pestaña 'Métricas Detalladas' contiene el dataGridViewMetricas con los resultados de cada una de las 800 ejecuciones individuales.

Columnas de la tabla:

- **Tamaño:** 100, 1000, 5000 o 10000
- **Iteración:** Número de iteración (1-5)
- **Algoritmo:** Nombre del algoritmo ejecutado
- **Tipo de datos:** Categoría del conjunto de prueba
- **Ticks:** Tiempo medido en ticks del procesador
- **Milisegundos:** Tiempo medido en milisegundos (4 decimales)
- **¿Orden correcto?:** 'Sí' o 'No' según la validación automática
- **Comparaciones:** Número de comparaciones entre elementos
- **Intercambios:** Número de intercambios de posición
- **Asignaciones:** Número de asignaciones de valores
- **Llamadas recursivas:** Contador para algoritmos recursivos

Validación: Si un algoritmo falla en ordenar correctamente, la fila se resalta en color rojo claro (LightCoral) para identificación inmediata.

4.5. ESTADÍSTICAS CONSOLIDADAS

La pestaña 'Estadísticas Consolidadas' muestra el dataGridViewEstadisticas con promedios, mínimos y máximos calculados a partir de las 5 iteraciones de cada combinación.

Columnas:

- **Tamaño, Algoritmo, Tipo de datos:** Identificadores de la prueba
- **Tiempo mínimo (ms):** Mejor tiempo de las 5 iteraciones
- **Tiempo máximo (ms):** Peor tiempo de las 5 iteraciones
- **Tiempo promedio (ms):** Promedio aritmético de las 5 iteraciones

Al presionar 'Generar Estadísticas', el sistema también ejecuta automáticamente el método GenerarConclusiones() que analiza los datos y presenta hallazgos en el textBoxConclusiones.

4.6. GRÁFICOS COMPARATIVOS

La pestaña 'Gráficos Comparativos' contiene el chartResultados con gráficos de columnas agrupadas.

Filtros:

- comboBoxGrafTamano: Filtra por tamaño de arreglo
- comboBoxGrafTipo: Selecciona el tipo de datos a graficar

El gráfico muestra en el eje X los 10 algoritmos, en el eje Y el tiempo promedio en milisegundos, y una serie por cada tamaño de entrada. Esto permite comparar visualmente cómo escala cada algoritmo al aumentar el tamaño del arreglo.

Recomendación: No se grafican todas las ejecuciones individuales para evitar saturación. Se trabaja con promedios de las 5 iteraciones.

5. DESCRIPCIÓN TÉCNICA

Esta sección describe la arquitectura interna de la aplicación, los algoritmos implementados y las métricas operacionales registradas.

5.1. ALGORITMOS IMPLEMENTADOS

1. Ordenamiento por Inserción

Complejidad: $O(n^2)$ peor caso, $O(n)$ mejor caso. Eficiente para arreglos pequeños o casi ordenados. En la implementación se cuenta cada comparación ($arr[j] > key$), cada intercambio y cada asignación.

2. Ordenamiento por Selección

Complejidad: $O(n^2)$ en todos los casos. Realiza siempre $n(n-1)/2$ comparaciones. El número de intercambios es como máximo $n-1$.

3. Ordenamiento de Burbuja

Complejidad: $O(n^2)$ peor y promedio, $O(n)$ mejor caso (con optimización no implementada aquí). Muy ineficiente para datos grandes. Útil como referencia de peor desempeño.

4. Shell Sort

Complejidad: Entre $O(n)$ y $O(n^2)$ dependiendo de la secuencia de gaps. Usa $gap = n/2$, luego $gap/2$. Mejor que Inserción para tamaños medianos.

5. Merge Sort

Complejidad: $O(n \log n)$ en todos los casos. Algoritmo estable basado en divide y vencerás. Requiere memoria adicional $O(n)$. Se cuentan las llamadas recursivas.

6. Heap Sort

Complejidad: $O(n \log n)$ en todos los casos. No requiere memoria adicional. Basado en la estructura de montículo (heap).

7. Quick Sort

Complejidad: $O(n \log n)$ promedio, $O(n^2)$ peor caso. Muy rápido en la práctica para datos aleatorios. Usa pivote en el último elemento. Puede degradar con datos repetidos.

8. Quick Sort de Tres Vías

Variante de Quick Sort que particiona en tres zonas: $< \text{pivote}$, $= \text{pivote}$, $> \text{pivote}$. Especialmente eficiente cuando hay muchos valores duplicados, evitando la degradación a $O(n^2)$.

9. Counting Sort

Complejidad: $O(n + k)$ donde k es el rango de valores. No es comparativo. Adaptado para valores negativos mediante desplazamiento ($arr[i] - \min$). Limitado por el tamaño del rango; si excede `int.MaxValue`, delega a `Array.Sort()`.

10. Radix Sort

Complejidad: $O(d \times (n + k))$ donde d es el número de dígitos. Adaptado para negativos separando valores en listas de negativos y no negativos, ordenando por valor absoluto y reconstruyendo. Usa Counting Sort como subrutina por cada dígito decimal.

5.2. MÉTRICAS OPERACIONALES

Además del tiempo de ejecución, la aplicación registra métricas internas que permiten analizar el comportamiento de cada algoritmo independientemente del hardware:

Comparaciones: Cada vez que se evalúa una condición entre dos elementos del arreglo (ej. `arr[j] > arr[j+1]`). Indica la carga de trabajo del algoritmo.

Intercambios: Cada vez que se intercambian dos posiciones del arreglo. Algunos algoritmos como Selección hacen pocos intercambios; Burbuja hace muchos.

Asignaciones: Cada asignación de valor a una posición del arreglo o variable auxiliar. Incluye copias en Merge Sort y Counting Sort.

Llamadas recursivas: Contador incrementado al inicio de cada función recursiva. Aplicable a Merge Sort, Quick Sort y Quick Sort 3 Vías.

Estas métricas son útiles porque complementan el análisis de tiempo: un algoritmo puede ser rápido en milisegundos pero realizar muchas comparaciones, o viceversa. Permiten entender el comportamiento interno sin depender de la velocidad del procesador.

5.3. VALIDACIÓN AUTOMÁTICA DEL ORDENAMIENTO

Después de cada ejecución, la aplicación verifica que el arreglo resultante esté ordenado de menor a mayor mediante el método `ValidarOrdenamiento()`:

Recorre el arreglo comparando cada elemento con su sucesor. Si encuentra `arr[i-1] > arr[i]`, retorna false. Si completa el recorrido, retorna true.

El resultado se almacena en el campo `OrdenCorrecto` de la estructura `Metrica` y se muestra en la columna '¿Orden correcto?' del `DataGridView`. Las filas con error se resaltan en rojo.

Esta validación es crítica porque garantiza que los tiempos registrados corresponden a ordenamientos correctos, descartando mediciones inválidas.

6. RESPUESTAS A LAS PREGUNTAS DEL TALLER

Esta sección responde las 12 preguntas planteadas en el documento del taller, basándose en el análisis teórico y los resultados esperados del experimento.

1. ¿Qué algoritmo presentó mejor tiempo promedio con datos aleatorios para 100, 1.000, 5.000 y 10.000 elementos?

Para arreglos pequeños ($n=100$), los algoritmos de complejidad $O(n^2)$ como Inserción pueden ser competitivos debido a la baja constante oculta y la simplicidad del código. Sin embargo, a medida que el tamaño crece, los algoritmos $O(n \log n)$ dominan claramente.

Resultados esperados:

- **$n=100$:** Inserción o Quick Sort pueden tener tiempos similares. La diferencia no es significativa.
- **$n=1.000$:** Quick Sort, Merge Sort y Heap Sort se separan claramente de los $O(n^2)$.
- **$n=5.000$ y $n=10.000$:** Counting Sort y Radix Sort (si el rango es acotado) son los más rápidos por ser $O(n)$. Entre los comparativos, Quick Sort y Merge Sort lideran. Heap Sort es ligeramente más lento por la sobrecarga del heap.

Conclusión: Para rangos acotados, Counting Sort y Radix Sort son los ganadores absolutos. Para rangos grandes, Quick Sort y Merge Sort son los mejores comparativos.

2. ¿Qué algoritmos conservaron buen desempeño al aumentar el tamaño de entrada?

Los algoritmos con complejidad $O(n \log n)$ o mejor mantienen un crecimiento controlado del tiempo:

- **Merge Sort:** $O(n \log n)$ garantizado. El tiempo crece de forma predecible y logarítmica.
- **Heap Sort:** $O(n \log n)$ garantizado. Similar a Merge Sort en estabilidad de desempeño.
- **Quick Sort:** $O(n \log n)$ promedio. Muy rápido en la práctica, aunque teóricamente puede degradar a $O(n^2)$ con pivotes malos.
- **Counting Sort y Radix Sort:** $O(n)$ y $O(d \times n)$ respectivamente. Escalan linealmente, siendo los mejores para tamaños grandes con rangos acotados.

Shell Sort también mejora significativamente respecto a Inserción, aunque no alcanza el rendimiento de los $O(n \log n)$ puros.

3. ¿Qué algoritmos empeoraron más claramente cuando se pasó de 100 a 10.000 elementos?

Los algoritmos de complejidad cuadrática $O(n^2)$ muestran el empeoramiento más drástico:

- **Burbuja:** El peor de todos. De ~0.01 ms a ~1000+ ms (factor de 100.000x teórico, aunque en la práctica la constante oculta lo hace ligeramente menor).
- **Selección:** Similar a Burbuja en comportamiento cuadrático. Siempre hace $n(n-1)/2$ comparaciones.
- **Inserción:** Mejor que Burbuja y Selección, pero aún $O(n^2)$. Con datos aleatorios grandes, el número de desplazamientos crece cuadráticamente.

Ratio de crecimiento esperado: De $n=100$ a $n=10.000$, el factor teórico es $(10.000/100)^2 = 10.000$. Es decir, un algoritmo $O(n^2)$ debería tardar aproximadamente 10.000 veces más. Los $O(n \log n)$ deberían tardar aproximadamente $(10.000 \times \log 10.000) / (100 \times \log 100) \approx 166$ veces más.

4. ¿Qué algoritmo mejoró claramente con datos parcialmente ordenados?

Inserción es el algoritmo que más se beneficia de datos parcialmente ordenados (tendencia ascendente). Su complejidad en el mejor caso es $O(n)$ cuando el arreglo ya está ordenado, ya que solo realiza una comparación por elemento y no necesita desplazamientos.

Con datos de 'tendencia ascendente', Inserción puede ser incluso más rápida que Quick Sort para tamaños pequeños y medianos, porque Quick Sort sigue particionando recursivamente aunque los datos ya estén casi ordenados.

Shell Sort también mejora notablemente con datos parcialmente ordenados, ya que su mecanismo de gaps aprovecha las subsecuencias casi ordenadas.

5. ¿Qué algoritmo tuvo peor desempeño con datos descendentes?

Inserción tiene su peor caso con datos en orden descendente. Cada nuevo elemento debe desplazarse hasta la posición inicial, realizando el máximo número de comparaciones e intercambios: $n(n-1)/2$ comparaciones y aproximadamente el mismo número de desplazamientos.

Quick Sort también sufre con datos descendentes si el pivote se elige como el último elemento, ya que todas las particiones quedan desbalanceadas (una sublista vacía y otra de $n-1$ elementos), degradando a $O(n^2)$. Sin embargo, en nuestra implementación el pivote es el último elemento, por lo que este efecto debería observarse claramente.

Burbuja también empeora con datos descendentes, realizando el máximo número de intercambios.

6. ¿Qué diferencias se observaron entre Quick Sort y Quick Sort de tres vías con datos repetidos?

Con datos de 'alta repetición' (solo 5 valores distintos), Quick Sort tradicional sufre una degradación significativa porque el pivote tiende a crear particiones muy desbalanceadas cuando hay muchos elementos iguales. Esto puede acercarlo a $O(n^2)$ en el peor caso.

Quick Sort de tres vías resuelve este problema particionando en tres zonas: elementos menores, iguales y mayores al pivote. Los elementos iguales al pivote quedan en la zona central y no se

procesan recursivamente, reduciendo drásticamente el número de llamadas recursivas y comparaciones.

Diferencias esperadas:

- Quick Sort 3 Vías debería ser significativamente más rápido (posiblemente 2x a 5x) con alta repetición.
- El número de llamadas recursivas de Quick Sort 3 Vías debería ser mucho menor.
- Con datos aleatorios sin repetición, ambos deberían tener desempeños similares.

7. ¿Qué ventajas y limitaciones presentaron Counting Sort y Radix Sort?

Ventajas:

- **Complejidad lineal $O(n)$:** Son los algoritmos teóricamente más rápidos cuando el rango de valores es acotado.
- **No comparativos:** No realizan comparaciones entre elementos, sino que usan las propiedades de los valores numéricos directamente.
- **Estables:** Mantienen el orden relativo de elementos iguales.
- **Counting Sort** es extremadamente simple y rápido para rangos pequeños.
- **Radix Sort** maneja rangos mayores procesando dígito por dígito.

Limitaciones:

- **Dependencia del rango:** Counting Sort requiere memoria $O(k)$ donde k es el rango (max-min). Si el rango es millones, consume demasiada memoria.
- **Solo enteros:** No funcionan directamente con datos de punto flotante o strings sin adaptación.
- **Valores negativos:** Requieren adaptación (desplazamiento para Counting Sort, separación de negativos para Radix Sort), lo que añade complejidad.
- **Memoria adicional:** Counting Sort necesita un arreglo auxiliar de tamaño k . Radix Sort necesita arreglos temporales por cada dígito.
- **Constantes ocultas:** Aunque son $O(n)$, las constantes pueden hacer que para n pequeño sean más lentos que Quick Sort.

8. ¿Qué tanto variaron los resultados entre las cinco iteraciones de una misma prueba?

Se espera cierta variabilidad entre iteraciones debido a:

- **Generación aleatoria:** Cada iteración genera arreglos diferentes, aunque con la misma distribución.
- **Planificación del SO:** Windows puede interrumpir el proceso, afectando los tiempos.
- **Cache del procesador:** El estado de la caché varía entre ejecuciones.
- **Garbage Collector:** Pausas impredecibles de .NET pueden afectar mediciones.

Observaciones esperadas: Los algoritmos deterministas (Inserción, Selección, Burbuja) con el mismo arreglo deberían tener tiempos muy similares. La variación principal viene de la generación aleatoria de datos. Los algoritmos con comportamiento dependiente de los datos (Quick Sort) pueden mostrar mayor variabilidad. El uso de promedios de 5 iteraciones mitiga estos efectos aleatorios.

9. ¿Las mediciones en ticks y milisegundos llevan a las mismas conclusiones?

Sí, en general llevan a las mismas conclusiones, pero con matices importantes:

- **Ticks:** Unidad más precisa (1 tick $\approx$ 100 ns en Windows). Útil para diferenciar algoritmos muy rápidos donde los milisegundos redondean a cero.
- **Milisegundos:** Más intuitivos para el usuario. Para algoritmos lentos (Burbuja con $n=10.000$), los milisegundos son más significativos.
- **Relación:** Ticks y milisegundos son proporcionales ($ms = ticks \times 1000 / Frequency$). El ranking de algoritmos debería ser idéntico en ambas unidades.
- **Diferencias menores:** Pueden aparecer por redondeo en la conversión, pero no cambian las conclusiones cualitativas sobre qué algoritmo es más rápido.

10. ¿Qué algoritmo podría considerarse más eficiente para conjuntos pequeños y cuál para conjuntos grandes?

Conjuntos pequeños ($n \leq 100$):

- **Inserción** es generalmente el más eficiente. Su simplicidad (pocos saltos de código, buena localidad de caché) supera la sobrecarga de algoritmos más complejos como Quick Sort o Merge Sort. Además, si los datos están parcialmente ordenados, su complejidad se acerca a $O(n)$.
- **Selección** también es simple pero siempre hace $O(n^2)$ comparaciones, por lo que Inserción es preferible.

Conjuntos grandes ($n \geq 5.000$):

- **Counting Sort o Radix Sort** si el rango es acotado. Son $O(n)$ y dominan claramente.
- **Quick Sort** es el mejor comparativo general para datos aleatorios grandes. Muy rápido en la práctica con buena localidad de caché.
- **Merge Sort** es preferible cuando se necesita estabilidad o cuando el peor caso de Quick Sort es inaceptable.
- **Heap Sort** es útil cuando la memoria es limitada (ordenamiento in-situ garantizado $O(n \log n)$).

11. ¿Por qué medir comparaciones e intercambios puede complementar el análisis del tiempo?

Las métricas operacionales complementan el tiempo por varias razones fundamentales:

- **Independencia del hardware:** El tiempo depende de la velocidad del procesador, la carga del sistema operativo, el estado de la caché y el garbage collector. Las comparaciones e intercambios son métricas puramente algorítmicas que no varían con el hardware.
- **Identificación de cuellos de botella:** Un algoritmo puede ser lento no por muchas comparaciones, sino por muchos intercambios (que implican escrituras en memoria). Por ejemplo, Burbuja hace muchos intercambios; Selección hace pocas comparaciones pero también pocos intercambios.

- **Predicción de escalabilidad:** Si un algoritmo tiene muchas comparaciones pero pocos intercambios, puede escalar mejor en sistemas donde las escrituras son costosas.
- **Validación teórica:** Las métricas permiten verificar empíricamente las complejidades teóricas. Por ejemplo, Selección debería tener aproximadamente $n(n-1)/2$ comparaciones, independientemente del input.
- **Análisis de estabilidad:** El número de asignaciones en Merge Sort refleja su costo de memoria adicional, algo que el tiempo solo no revela claramente.

12. ¿Qué decisiones de diseño de interfaz ayudaron a organizar una cantidad alta de resultados?

Las siguientes decisiones de diseño fueron clave para manejar 800 ejecuciones sin saturar al usuario:

- **TabControl con 5 pestañas:** Separa claramente las funcionalidades: configuración, datos, métricas, estadísticas y gráficos. El usuario navega intencionalmente entre ellas.
- **GroupBox organizadores:** Dentro de cada pestaña, los GroupBox agrupan controles relacionados (parámetros, ejecución, progreso, filtros), reduciendo la carga cognitiva.
- **Filtros con ComboBox:** En lugar de mostrar todos los datos simultáneamente, el usuario selecciona tamaño, iteración y tipo de datos. Esto reduce la información mostrada a lo relevante.
- **Limitación de filas en DataGridView:** La visualización de datos generados se limita a 100 filas para evitar congelamiento de la interfaz con arreglos de 10.000 elementos.
- **Gráficos con promedios:** No se grafican las 800 ejecuciones individuales, sino promedios por combinación. Esto produce gráficos limpios con 10 barras por serie.
- **Barra de progreso y etiqueta:** Durante las 800 ejecuciones, el usuario ve el progreso (X/800) y un porcentaje, evitando la sensación de que la aplicación se congeló.
- **Conclusiones automáticas en TextBox:** En lugar de obligar al usuario a interpretar 800 filas, el sistema extrae automáticamente los hallazgos más importantes y los presenta en texto plano.
- **Colores de validación:** Las filas con errores de ordenamiento se resaltan en rojo, permitiendo identificar problemas sin leer cada celda.
- **Botones deshabilitados durante ejecución:** Evitan que el usuario interactúe accidentalmente mientras el proceso experimental está en curso.

7. CONCLUSIONES

El desarrollo de esta aplicación permitió consolidar los conceptos de análisis de algoritmos vistos en clase, relacionando la teoría de complejidad computacional con resultados empíricos medidos en un entorno real.

Principales hallazgos:

1. La complejidad teórica se confirma empíricamente: los algoritmos $O(n^2)$ degradan drásticamente al aumentar el tamaño, mientras que los $O(n \log n)$ mantienen un crecimiento controlado.
2. Counting Sort y Radix Sort demuestran que los algoritmos no comparativos pueden superar significativamente a los comparativos cuando las condiciones son favorables (rangos acotados).
3. Quick Sort de tres vías es una mejora sustancial sobre Quick Sort tradicional cuando hay muchos valores duplicados, evitando la degradación a $O(n^2)$.
4. Inserción, aunque teóricamente $O(n^2)$, es práctica y eficiente para conjuntos pequeños o parcialmente ordenados, lo que explica su uso en implementaciones híbridas (Timsort, Introsort).
5. Las métricas operacionales (comparaciones, intercambios, asignaciones) proporcionan una visión más profunda del comportamiento interno que el tiempo solo no puede ofrecer.
6. El diseño experimental automatizado (800 ejecuciones controladas) garantiza resultados estadísticamente significativos y reproducibles.

Lecciones de diseño de software: La organización de la interfaz en pestañas con filtros selectivos fue esencial para presentar una gran cantidad de datos sin abrumar al usuario. La validación automática de ordenamiento garantiza la integridad de los resultados experimentales.

8. ANEXOS

8.1. ESPECIFICACIONES DEL DISEÑO EXPERIMENTAL

El diseño experimental sigue estrictamente las especificaciones del taller:

Variable	Valores
Tamaños de arreglo	100, 1.000, 5.000, 10.000
Iteraciones por tamaño	5 independientes
Tipos de datos	Aleatorio, Tendencia ascendente, Descendente, Alta repetición
Algoritmos	10 (Inserción, Selección, Burbuja, Shell, Merge, Heap, Quick, Quick 3V, Counting, Radix)
Total de ejecuciones	$4 \times 5 \times 4 \times 10 = 800$
Métricas de tiempo	Ticks (Stopwatch.ElapsedTicks) y Milisegundos (Stopwatch.Elapsed.TotalMilliseconds)
Métricas operacionales	Comparaciones, Intercambios, Asignaciones, Llamadas recursivas
Validación	Algoritmo de verificación de orden ascendente automático

8.2. COMPLEJIDADES TEÓRICAS DE LOS ALGORITMOS

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio auxiliar	Estable
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Shell Sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$	$O(1)$	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Quick Sort 3V	$O(n \log n)$	$O(n \log n)$	$O(n \log n)^*$	$O(\log n)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Sí
Radix Sort	$O(d \times (n+k))$	$O(d \times (n+k))$	$O(d \times (n+k))$	$O(n+k)$	Sí

* Quick Sort de tres vías reduce la probabilidad del peor caso $O(n^2)$ con datos repetidos, aunque teóricamente sigue siendo posible.

8.3. ESTRUCTURAS DE DATOS PRINCIPALES

La aplicación utiliza las siguientes estructuras para almacenar resultados:

struct Metrica: Almacena los resultados de una ejecución individual (tamaño, iteración, algoritmo, tipo de datos, ticks, milisegundos, validación, y las 4 métricas operacionales).

struct Estadistica: Almacena los promedios consolidados por combinación de tamaño, algoritmo y tipo de datos (mínimo, máximo, promedio de los 5 tiempos).

Dictionary<string, int[][]> datosGenerados: Almacena los arreglos originales usando como clave 'tamano_iteracion'. Cada entrada contiene 4 arreglos (uno por tipo de datos).

List<Metrica> listaMetricas: Lista con las 800 métricas individuales.

List<Estadistica> listaEstadisticas: Lista con las estadísticas consolidadas.

8.4. LISTA DE CONTROLES UTILIZADOS (Diseñador Visual)

Todos los controles fueron creados en el diseñador visual de Visual Studio (Form1.Designer.cs), cumpliendo con la restricción del taller de no generar controles por código:

Control	Nombre	Uso
TabControl	tabControlPrincipal	Organización en 5 pestañas
TabPage	tabPageConfiguracion	Pestaña de configuración
TabPage	tabPageDatos	Pestaña de datos generados
TabPage	tabPageMetricas	Pestaña de métricas detalladas
TabPage	tabPageEstadisticas	Pestaña de estadísticas
TabPage	tabPageGraficos	Pestaña de gráficos comparativos
GroupBox	groupBoxParametros	Agrupar controles de entrada
GroupBox	groupBoxEjecucion	Agrupar botones de control
GroupBox	groupBoxProgreso	Agrupar barra de progreso
GroupBox	groupBoxResultados	Agrupar conclusiones
GroupBox	groupBoxFiltrosDatos	Agrupar filtros de datos
GroupBox	groupBoxFiltrosGraficos	Agrupar filtros de gráficos
NumericUpDown	numericUpDownMin	Entrada de valor mínimo
NumericUpDown	numericUpDownMax	Entrada de valor máximo
Button	buttonEjecutar	Inicia las 800 pruebas
Button	buttonVerDatos	Navega a datos generados

Button	buttonGenerarGraficos	Genera gráficos comparativos
Button	buttonGenerarEstadisticas	Calcula estadísticas
Button	buttonExportarPDF	Exporta resultados a texto
ProgressBar	progressBarEjecucion	Muestra progreso visual
Label	labelProgreso	Texto de progreso X/800
Label	labelMin	Etiqueta 'Valor mín.'
Label	labelMax	Etiqueta 'Valor máx.'
Label	labelInfoRangos	Información de rangos
Label	labelConclusiones	Etiqueta de conclusiones
Label	labelTamano	Etiqueta 'Tamaño'
Label	labelIteracion	Etiqueta 'Iteración'
Label	labelTipoDatos	Etiqueta 'Tipo de datos'
Label	labelGrafTamano	Etiqueta filtro gráfico tamaño
Label	labelGrafTipo	Etiqueta filtro gráfico tipo
ComboBox	comboBoxTamano	Selección de tamaño (100-10000)
ComboBox	comboBoxIteracion	Selección de iteración (1-5)
ComboBox	comboBoxTipoDatos	Selección de tipo de datos
ComboBox	comboBoxGrafTamano	Filtro gráfico por tamaño
ComboBox	comboBoxGrafTipo	Filtro gráfico por tipo
DataGridView	dataGridViewDatos	Tabla de datos generados
DataGridView	dataGridViewMetricas	Tabla de métricas detalladas
DataGridView	dataGridViewEstadisticas	Tabla de estadísticas
Chart	chartDatos	Gráfico de datos originales
Chart	chartResultados	Gráfico comparativo de tiempos
TextBox	textBoxConclusiones	Área de conclusiones automáticas

8.5. GLOSARIO DE TÉRMINOS

Tick: Unidad mínima de tiempo medible por el procesador. En Windows, 1 tick $\approx$ 100 nanosegundos.

Stopwatch: Clase de .NET para medir intervalos de tiempo de alta precisión.

Complejidad temporal: Medida del crecimiento del tiempo de ejecución respecto al tamaño de entrada.

Complejidad espacial: Medida del crecimiento de la memoria adicional requerida.

Algoritmo estable: Mantiene el orden relativo de elementos con claves iguales.

Ordenamiento in-situ: Ordena el arreglo sin requerir memoria adicional significativa.

Divide y vencerás: Estrategia que divide el problema en subproblemas más pequeños.

Pivote: Elemento de referencia usado en Quick Sort para particionar el arreglo.

Montículo (Heap): Estructura de árbol binario completo que satisface la propiedad del montículo.

Gap (Shell Sort): Distancia entre elementos comparados en cada pasada del algoritmo.

Documento generado para el Taller Práctico Final de Matemáticas Discretas - UPTC Sogamoso. Todos los controles de la interfaz fueron creados en el diseñador visual de Visual Studio.